

COMP1549: Advanced Programming

Generics

Dr Taimoor Khan

WWW

- [Personal](#)
- Operational Lead - NCSC accredited [ACE-CSR](#)
- Deputy Director - NCSC accredited [ACE-CSE](#)
- [RITICS Fellow](#) – Imperial College London

5th February 2026 - Week 4

Generics

Generics - the problem

- Prior to the introduction of Generics

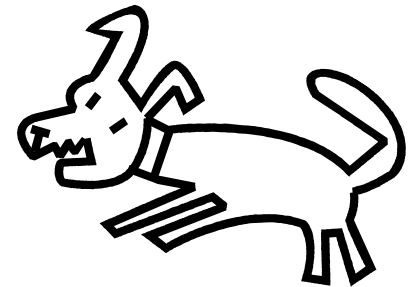
```
List catList = new ArrayList();  
catList.add(new Cat("fluffy"));  
catList.add(new Cat("tigger"));  
catList.add(new Dog("gruffly"));
```

.....

```
Cat mog1 = (Cat)catList.get(0);  
Cat mog2 = (Cat)catList.get(1);  
Cat mog3 = (Cat)catList.get(2);
```



// woof woof



Generics - the solution

- Introduced in Java SE 5.0

```
List<Cat> moreMogs = new ArrayList<Cat>();  
moreMogs.add(new Cat("tom"));  
moreMogs.add(new Cat("felix"));  
moreMogs.add(new Dog("gnasher")); // compiler error
```

.....

```
Cat mog4 = moreMogs.get(0); // note that no casts ...  
Cat mog5 = moreMogs.get(1); // ... are needed
```

The diamond notation

```
List<Cat> moreMogs = new ArrayList<>();  
moreMogs.add(new Cat("tom"));  
moreMogs.add(new Cat("felix"));  
moreMogs.add(new Dog("gnasher")); // compiler error  
  
.....  
Cat mog4 = moreMogs.get(0); // note that no casts ...  
Cat mog5 = moreMogs.get(1); // ... are needed
```

Creating our own generic classes

- The previous example shows how we can **use** generic classes from the Java API but we can also create our own.
- It's difficult to come up with simple but useful examples so forgive the trivial example coming up.
- Following example based on Schildt chapter 3.

Example - Developing a Generic class

```
class Gen<T> {  
    T ob;  
    Gen(T o) {  
        ob = o;  
    }  
    T getob() {  
        return ob;  
    }  
    void showType() {  
        System.out.println("Type of T is " +  
            ob.getClass().getName());  
    }  
}
```

Declare a variable of type **T** - whatever it is

Constructor expects and Object of type **T** - whatever it is

Return type is type **T**

Method that displays the class of **T**

GenDemo.java (continued on next slide)

... and using it

```
class GenDemo {  
    public static void main(String args[]) {  
        Gen<Integer> iOb;  
        iOb = new Gen<>(88);  
        iOb.showType();  
        int v = iOb.getob();  
        System.out.println("value: " + v);  
        System.out.println();  
        Gen<String> strOb = new Gen<>("Generics Test");  
        strOb.showType();  
        String str = strOb.getob();  
        System.out.println("value: " + str);  
    }  
}
```

declare an Integer version and give it a value of 88

outputs
"Type of T is java.lang.Integer"

note that there is no need to cast

a String version

outputs
"Type of T is java.lang.String"

again no need to cast

```
class WeirdGen<X, Y> {
```

```
    X xOne, xTwo;
```

```
    Y yOne;
```

```
    public WeirdGen (X x1, X x2, Y y1) {
```

```
        xOne = x1;
```

```
        xTwo = x2;
```

```
        yOne = y1;
```

```
    }
```

```
    public void displayWeird() {
```

```
        System.out.println(xOne.toString()
```

```
            + yOne.toString()
```

```
            + xTwo.toString());
```

```
    }
```

```
}
```

```
public class TestWeirdGen {
```

```
    public static void main(String[] args) {
```

```
        WeirdGen<String, String, Integer> wg1;
```

```
        wg1 = new WeirdGen<>("I O U ", " pounds", 100);
```

```
        wg1.displayWeird();
```

```
    }
```

```
}
```

TestWeirdGen.java

correct syntax for creating a
generic class with two type
parameters. Remember that
you can call them what you
like: X, Y, T or whatever

Do you think this will compile?

If not, why not.

If so, what will it output?

Something you can't do

- You can't create an instance of a parameterized type

```
class Gen<T> {  
    T ob1, ob2;  
    Gen(T o) {  
        ob1 = o;  
        ob2 = new T(); // compiler error  
    }  
    T getob1() {  
        return ob1;  
    }  
    T getob2() {  
        return ob2;  
    }  
}
```

CantDoltDemo.java

Type parameters must match EXACTLY

```
Gen<Double> dOb = new Gen<Integer>(44);
```

```
Gen<Object> oOb = new Gen<String>("xxx");
```

- Both the above give compilations errors
- The type parameter must match **exactly**.
- Even though String extends Object we can't use a Gen<String> where a Gen<Object> is expected

Raw types

- For compatibility with pre-Java 5 code, you are allowed to use a generic class without giving the type argument.
- This is called using a *raw* type e.g.



```
class Gen<T> {  
    T ob;  
    Gen(T o) { ob = o; }  
    T getob() { return ob; }  
}
```

```
Gen<Integer> iOb = new Gen<>(88);  
Gen<String> strOb =  
    new Gen<>("Generics Test");
```

```
Gen raw = new Gen(98.6);
```



- For raw types the type defaults to Object
- Should only be done when absolutely necessary as it is **dangerous**.

```
class RawDemo {  
    public static void main(String args[]) {
```

```
        Gen<String> strOb = new Gen<>("Generics Test");  
        Gen raw = new Gen(98.6);
```

```
        // Cast here is necessary because type is unknown.
```

```
        double d = (Double) raw.getob();  
        System.out.println("value: " + d);
```

```
        String s = (String) raw.getob();
```

```
        strOb = raw;
```

Will this compile?

```
        String str = strOb.getob();
```

If not, why not?

If it will compile what will be
output?

```
    }
```

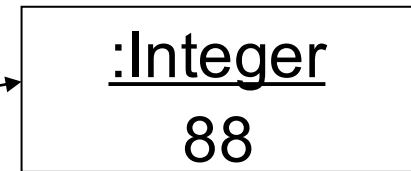
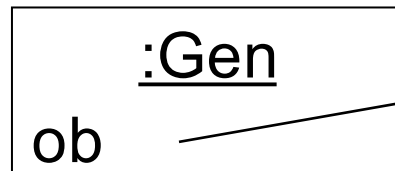
RawDemo.java

```
}
```

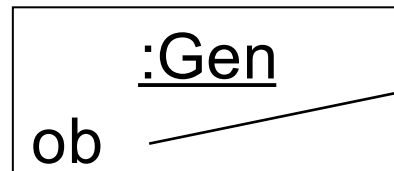
Not knowing the type is limiting

- A problem with the preceding examples is that it is difficult for the generic class to do anything useful because the code in it can't assume anything about the type parameters.

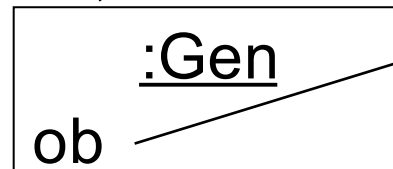
Gen<Integer> iOb;



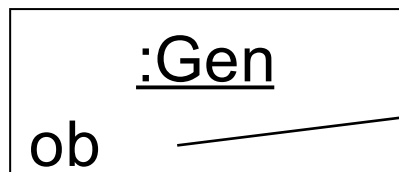
Gen<String> strOb;



Gen <InsurancePolicy> ipolOb;



Gen <FlyingPig> pigOb;



Knowing nothing about parameter type

```
class Stats1<T> {  
    T[] nums; // nums is an array of type T  
  
    Stats1(T[] o) {  
        nums = o;  
    }  
  
    // Return type double in all cases.  
    public double average() {  
        double sum = 0.0;  
  
        for(T num : nums)  
            sum += num.doubleValue(); // Error!!!  
  
        return sum / nums.length;  
    }  
}
```

Trying to create a class that takes an array of any numeric type and produces some statistics e.g., the average

But because the type is unknown, we can't call `doubleValue()` to get the number to be able to do arithmetic on it.

BoundsDemo1.java

Bounded types to the rescue

- Bounded types are where the type of the type parameter is limited by an upper bound

```
class Stats1<T> {
```

```
.....
```

T can be any type from
Object down e.g.

```
Stats1<String> myStats;
```

```
class Stats1<T extends Number> {
```

```
.....
```

T can only be class Number or a
class that inherits from class
Number e.g.

```
Stats1<Integer> myStats;
```

```
class Stats2<T extends Number> {  
    T[] nums; // array of Number or subclass
```

```
    // Pass the constructor a reference to  
    // an array of type Number or subclass.
```

```
    Stats2(T[] o) {  
        nums = o;  
    }
```

```
    // Return type double in all cases.
```

```
    double average() {  
        double sum = 0.0;
```

```
        for(T num : nums)  
            sum += num.doubleValue();
```

```
        return sum / nums.length;
```

BoundsDemo2.java

class Number has the
method doubleValue()

so whatever T is it must
have doubleValue() too

so now this is allowed

// Demonstrate Stats2.

```
class BoundsDemo2 {  
    public static void main(String args[]) {
```

```
        Integer inums[] = { 1, 2, 3, 4, 5 };  
        Stats2<Integer> iob = new Stats2<>(inums);  
        double v = iob.average();  
        System.out.println("iob average is " + v);
```

```
        Double dnums[] = { 1.1, 2.2, 3.3, 4.4, 5.5 };  
        Stats2<Double> dob = new Stats2<>(dnums);  
        double w = dob.average();  
        System.out.println("dob average is " + w);
```

// This won't compile because String is not a
// subclass of Number.

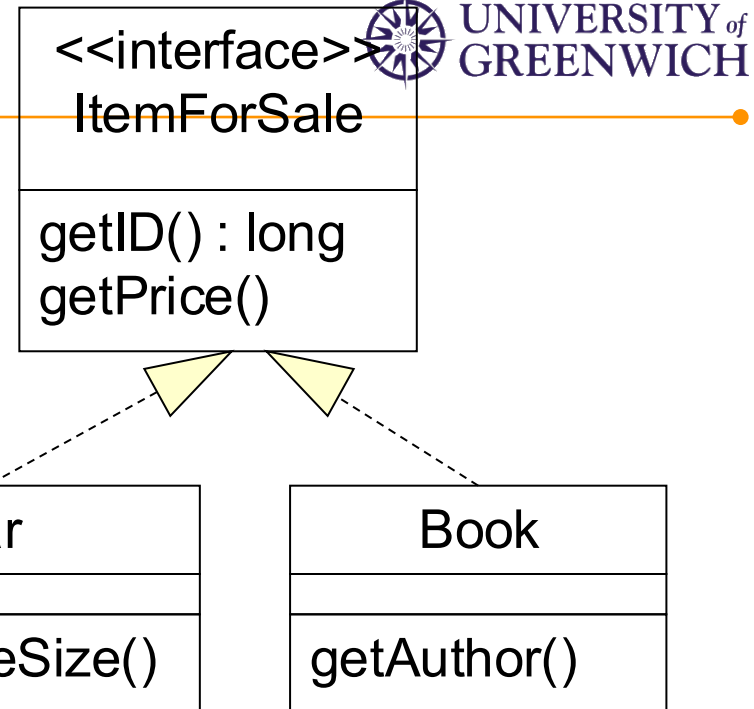
```
        String strs[] = { "1", "2", "3", "4", "5" };  
        Stats2<String> strob = new Stats2<>(strs);  
        double x = strob.average();  
        System.out.println("strob average is " + x);
```

BoundsDemo2.java

```

class Shop<T extends ItemForSale> {
    Map<Long, T> stock;
    Shop() { stock = new HashMap<>(); }
    public void addToStock(T item) {
        stock.put(item.getID(), item);
    }
    public String getBookAuthor(long id) {
        return stock.get(id).getAuthor();
    }
    public float getItemPrice(long id) {
        return stock.get(id).getPrice();
    }
    .....
}

```



which lines won't compile and why?

```

Shop<Car> carShowRoom = new Shop<>();
Shop<Book> bookShop = new Shop<>();
Shop<String> stringShop = new Shop<>();

carShowRoom.addToStock(new Car(2378964, 8000.00F, 1200));
bookShop.addToStock(new Book(1676008945, 29.50F, "Smith"));
carShowRoom.addToStock(new Book(1536008765, 9.99F, "Haynes"));

```

When to use Generics

1. Whenever you are using a generic class specify the type parameter(s) rather than using the raw type.
2. When creating your own class you may want/need to make it generic if ...
 - it inherits from a generic class e.g.
`class MyArrayList<T> extends ArrayList<T> {`
 - it makes use of a generic class e.g.
`class photoAlbum<X, Y> {
 private HashMap<X, Y> theAlbum;`
 - there's some other good reason to do so :-)

Java annotations

- Annotations are metadata, i.e., provide information about program but is not part of the actual program
 - Information for the compiler
 - ❖ Can be used by the compiler to detect and suppress errors
 - Compile-time and deployment-time processing
 - ❖ Can be used by tools to generate code, XML files and configurations
 - Runtime processing
 - ❖ Can be used to monitor program execution

Generics and annotations

- Introduced in Java 8 and later
- Used to annotate type parameters

```
public< @Actionable T > void performAction( final T action )  
    { // Some implementation here }  
  
final Collection< @NotEmpty String > strings = new ArrayList<>();
```

Annotations as interfaces

- Annotations can be user defined, i.e., can be programmed

```
public @interface SimpleAnnotation { }
```

- The @interface keyword introduces new annotation type, that is why they are called specialized interfaces. Annotations may declare the attributes with or without default values.

```
public @interface SimpleAnnotationWithAttributes  
{  
    String name();  
    int order() default 0;  
}
```

- Annotations can be instantiated as

```
@SimpleAnnotationWithAttributes( name = "new annotation" )
```

Annotations processing

- ◉ Annotations are syntactic sugar for metadata
- ◉ Every annotation has a property of retention policy (of type RetentionPolicy)
 - That defines rules how to retain annotations
 - ❖ CLASS
 - Annotations are to be recorded in the class by the compiler but **may not** be retained by VM at run-time
 - ❖ RUNTIME
 - Annotations are to be recorded in the class by the compiler and **are** retained by VM at run-time
 - ❖ SOURCE
 - Annotations are to be discarded by the compiler

Example - retention policy

```
import java.lang.annotation.Retention;  
import java.lang.annotation.RetentionPolicy;  
@Retention( RetentionPolicy.RUNTIME )  
public @interface AnnotationWithRetention { }
```

- The above annotations are enforced to be retained at run-time by VM

Annotations and element types

- Each annotations has an element type it could be applied to, e.g.,
 - ANNOTATION_TYPE
 - CONSTRUCTOR
 - FIELD
 - LOCAL_VARIABLE
 - METHOD
 - PACKAGE
 - PARAMETER
 - TYPE

Example – element types

- In contrast to retention policy, annotations can be associated with multiple element types

```
import java.lang.annotation.ElementType;  
import java.lang.annotation.Target;  
@Target( { ElementType.FIELD, ElementType.METHOD } )  
public @interface AnnotationWithTarget {}
```

Limitations of Generics

- Primitive types (like `int`, `long`, `byte`, . . .) are not allowed to be used in generics
 - Only their wrapper classes can be used (like `Integer`, `Long`, `Byte`, ...)
 - That results in implicit boxing/unboxing
- ```
final List< Long > longs = new ArrayList<>();
longs.add(0L); // 'long' is boxed to 'Long'

long value = longs.get(0); // 'Long' is unboxed to 'long'
```

# Limitations of Generics

- Generics erase the types

- Note that generics exist only at compile time
- JVM compiled code has only concrete types erased, e.g., following code does not compile

```
void sort(Collection< String > strings)
{ // Some implementation over strings here }
```

```
void sort(Collection< Number > numbers)
{ // Some implementation over numbers here }
```

- Two methods are narrowed down to the same signature and leads to compilation error

```
void sort(Collection strings)
```

```
void sort(Collection numbers)
```

# Limitations of Generics

- It is also not possible to create the array instances using generics
  - Following code does not compile

```
public< T > void performAction(final T action)
{ T[] actions = new T[0]; }
```

# End of week 4!